Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par :

Yassine Meftah

Année Universitaire : 2025-2026

Résumé

Ce projet s'inscrit dans le contexte de l'analyse comportementale de la clientèle d'une entreprise de e-commerce de cadeaux. L'objectif principal est de construire une chaîne complète de Machine Learning capable de prédire le churn client, d'explorer la structure des données, de segmenter les profils clients et de proposer une solution d'inférence réutilisable. Pour cela, un pipeline complet a été mis en place : préparation des données, parsing des dates, feature engineering, détection et correction des valeurs aberrantes, suppression des colonnes redondantes ou fuyantes, rééquilibrage des classes par SMOTE/SMOTENC, réduction de dimension par ACP, clustering KMeans, classification supervisée, régression et validation croisée. Le meilleur modèle de classification obtenu est une régression logistique optimisée, avec des performances très élevées en accuracy, F1-score et ROC-AUC. Une phase d'inférence a également été développée via un script de prédiction et une interface Flask. Le projet met ainsi en œuvre une démarche complète, reproductible et proche d'un cas réel de mise en production.

Abstract

This project focuses on customer behavioral analysis in a retail e-commerce context. The main objective is to build a complete Machine Learning pipeline capable of predicting customer churn, exploring data structure, segmenting customer profiles, and providing a reusable inference solution. To achieve this, a full workflow was implemented : data preparation, date parsing, feature engineering, outlier handling, removal of redundant or leakage-prone variables, class rebalancing with SMOTE/SMOTENC, dimensionality reduction with PCA, KMeans clustering, supervised classification, regression, and cross-validation. The best classification model is an optimized logistic regression with excellent performance in terms of accuracy, F1-score, and ROC-AUC. An inference phase was also developed through a prediction script and a Flask interface. Overall, this project demonstrates a complete, reproducible workflow close to a real deployment scenario.

Table des matières

1 Introduction générale **4**
1.1 Contexte 4
1.2 Problématique 4
1.3 Objectifs du projet 4
1.4 Démarche suivie 4

2 Présentation du jeu de données **4**
2.1 Description générale 4
2.2 Typologie des variables 4
2.3 Problèmes de qualité identifiés 5

3 Analyse exploratoire et compréhension des données **5**
3.1 Qualité des données 5
3.2 Corrélations et multicolinéarité 5
3.3 Déséquilibre de classes 6
3.4 Lecture métier 6

4 Préparation des données **6**
4.1 Nettoyage initial . . . 7
4.2 Parsing et transformation des dates . . . 7
4.3 Feature engineering . . . 7
4.4 Suppression de variables redondantes . . . 7
4.5 Gestion des outliers . . . 7
4.6 Gestion des valeurs manquantes . . . 8
4.7 Suppression de variables . . . 8
4.8 Encodage et normalisation . . . 8
4.9 Rééquilibrage du dataset . . . 8
4.10 Séparation train/test . . . 8

5 Réduction de dimension et segmentation **8**
5.1 Analyse en composantes principales (PCA) . . . 8
5.2 Interprétation de la PCA . . . 9
5.3 KMeans . . . 9
5.4 Choix final du nombre de clusters . . . 10
5.5 Interprétation des clusters . . . 10
5.6 Limites de la segmentation . . . 11

6 Modélisation prédictive : classification **11**
6.1 Objectif . . . 11
6.2 Modèles testés . . . 11
6.3 Optimisation des hyperparamètres . . . 11
6.4 Métriques d'évaluation . . . 12
6.5 Résultats obtenus . . . 12
6.6 Matrice de confusion . . . 12
6.7 Analyse critique . . . 13

7 Modélisation prédictive : régression **13**
7.1 Objectif . . . 13
7.2 Modèles testés . . . 13
7.3 Métriques . . . 13
7.4 Résultats . . . 13
7.5 Analyse critique . . . 13

8 Validation croisée et robustesse **14**
8.1 Validation croisée en classification . . . 14
8.2 Validation croisée en régression . . . 14
8.3 Interprétation . . . 14

9 Prédiction et inférence **14**
9.1 Rôle de `predict.py` . . . 14
9.2 Pipeline d'inférence . . . 14
9.3 Test sur données complètes . . . 15
9.4 Limite observée . . . 15

10 Déploiement et mise en production avec Flask **15**
10.1 Objectif du déploiement 15
10.2 Architecture de la solution 15
10.3 Deux modes de test 15
10.4 Fonctionnement détaillé 15
10.5 Problèmes rencontrés et corrections 16
10.6 Résultats observés dans l'interface 16
10.7 Interprétation métier des probabilités 18
10.8 Limites du déploiement actuel 18
10.9 Améliorations possibles 18

11 Discussion générale **19**
11.1 Points forts du projet 19
11.2 Limites 19
11.3 Améliorations possibles 19

12 Conclusion générale **19**

1 Introduction générale

1.1 Contexte

Dans le domaine du retail et du e-commerce, l'analyse des données clients constitue un levier stratégique pour améliorer la fidélisation, personnaliser les campagnes marketing et optimiser la rentabilité. La capacité à identifier les clients susceptibles de quitter la plateforme permet d'anticiper les actions commerciales et de réduire la perte de valeur.

1.2 Problématique

Le churn client représente un enjeu majeur pour les entreprises, car il est généralement plus coûteux d'acquérir un nouveau client que de conserver un client existant. Le problème traité dans ce projet consiste donc à exploiter les données comportementales et transactionnelles afin de prédire si un client risque de partir.

1.3 Objectifs du projet

Le projet poursuit plusieurs objectifs complémentaires :

- développer un modèle de classification pour prédire le churn ;
- analyser la structure des données et identifier les variables importantes ;
- appliquer une réduction de dimension par ACP pour la visualisation et le clustering ;
- segmenter les clients via KMeans ;
- entraîner également un modèle de régression pour prédire `MonetaryTotal` ;
- déployer une solution d'inférence réutilisable via un script Python et une interface Flask.

1.4 Démarche suivie

La méthodologie suivie s'organise selon une chaîne complète :

Préparation des données → Transformation → Modélisation → Évaluation → Déploiement

2 Présentation du jeu de données

2.1 Description générale

Le dataset étudié correspond à une base client retail contenant des variables numériques, catégorielles et dérivées. Il inclut des informations transactionnelles, temporelles, démographiques et comportementales. La variable cible principale du projet est `Churn`.

2.2 Typologie des variables

Les variables utilisées peuvent être classées en quatre groupes :

- **Variables transactionnelles :** `Frequency`, `MonetaryTotal`, `AvgBasketValue`, `ReturnRatio`, `UniqueProducts`, etc.
- **Variables temporelles :** `CustomerTenureDays`, `FirstPurchaseDaysAgo`, `PreferredHour`, `PreferredMonth`, etc.

— **Variables démographiques et catégorielles :** `Age`, `Gender`, `Country`, `Region`, `AccountStatus`.
— **Variables dérivées :** `RegYear`, `RegMonth`, `RegDay`, `RegWeekday`, `IsPrivateIP`, `IpFirstOctet`, `AvgBasketValue`.

2.3 Problèmes de qualité identifiés

L'analyse du dataset a mis en évidence plusieurs problèmes classiques :

— valeurs manquantes, notamment sur `Age` ;
— formats de date incohérents pour `RegistrationDate` ;
— présence d'outliers sur certaines colonnes numériques ;
— variables inutiles ou constantes ;
— variables potentiellement fuyantes vis-à-vis de la cible ;
— variables redondantes ou fortement corrélées.

3 Analyse exploratoire et compréhension des données

3.1 Qualité des données

La première étape a consisté à examiner la qualité globale du jeu de données, notamment la présence de valeurs manquantes, de colonnes constantes, de formats incohérents et de variables peu informatives. Cette phase a servi à guider les choix de nettoyage et de transformation.

3.2 Corrélations et multicolinéarité

Une analyse des corrélations entre variables numériques a été réalisée afin d'identifier les redondances potentielles. En complément, le facteur d'inflation de variance (VIF) a été calculé pour détecter la multicolinéarité sévère.

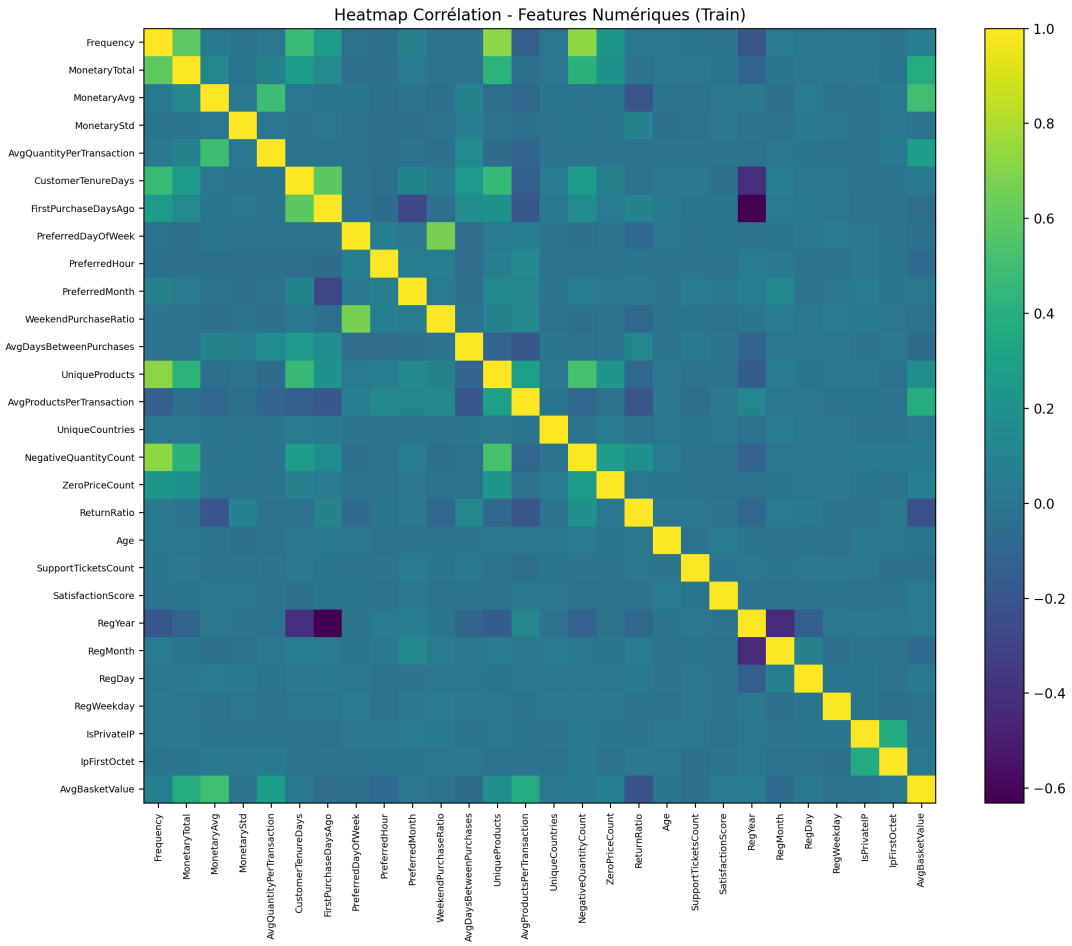

FIGURE 1 – Heatmap de corrélation entre les variables numériques.

L'analyse met en évidence plusieurs groupes de variables liées entre elles, notamment les variables transactionnelles et certaines variables dérivées. Cette lecture a justifié la suppression de certaines colonnes redondantes et l'application d'un filtrage automatique sur les corrélations élevées.

3.3 Déséquilibre de classes

La variable `Churn` présente un déséquilibre entre la classe des clients fidèles et celle des clients partants. Ce déséquilibre justifie le recours à une stratégie de rééquilibrage par sur-échantillonnage sur le train.

3.4 Lecture métier

Certaines variables ont une interprétation métier directe :

- `Frequency` mesure l'intensité d'activité ;
- `MonetaryTotal` et `AvgBasketValue` traduisent la valeur économique du client ;
- `SupportTicketsCount` et `SatisfactionScore` reflètent la relation client ;
- `CustomerTenureDays` et `FirstPurchaseDaysAgo` caractérisent l'ancienneté.

4 Préparation des données

4.1 Nettoyage initial

Plusieurs colonnes ont été supprimées dès le début du pipeline :

- `NewsletterSubscribed`, car il s'agit d'une variable sans variance utile ;
- `CustomerID`, car il s'agit d'un identifiant ;
- `ChurnRiskCategory` et toute colonne contenant `churnrisk`, car elles risquent d'introduire une fuite d'information ;
- `Recency` et certaines variables dérivées associées, afin d'éviter que le modèle n'apprenne directement une information trop proche de la cible.

4.2 Parsing et transformation des dates

La variable `RegistrationDate` a été convertie en type date puis décomposée en plusieurs composantes :

- `RegYear`
- `RegMonth`
- `RegDay`
- `RegWeekday`

4.3 Feature engineering

Des variables supplémentaires ont été construites à partir des données existantes :

- `IsPrivateIP` et `IpFirstOctet` à partir de `LastLoginIP` ;
- `AvgBasketValue` à partir de `MonetaryTotal` et `Frequency`.

D'autres variables dérivées comme `TenureRatio` ou `MonetaryPerDay` ont été volontairement exclues lorsqu'elles risquaient d'introduire du leakage.

4.4 Suppression de variables redondantes

Une partie du travail final a consisté à identifier et supprimer des colonnes redondantes ou dupliquant une information déjà portée par d'autres variables. Les colonnes suivantes ont été supprimées :

- `CancelledTransactions`
- `UniqueInvoices`
- `UniqueDescriptions`
- `AvgLinesPerInvoice`
- `MonetaryMin`
- `MonetaryMax`
- `MinQuantity`

Ce choix repose à la fois sur l'analyse statistique et sur une lecture métier visant à conserver les variables les plus interprétables.

4.5 Gestion des outliers

Les valeurs aberrantes ont été traitées via la méthode IQR. Les bornes ont été calculées uniquement sur le jeu d'entraînement puis réappliquées au jeu de test et au script d'inférence, afin d'éviter toute fuite d'information.

4.6 Gestion des valeurs manquantes

La stratégie retenue combine plusieurs méthodes :

— **médiane** pour la plupart des variables numériques ;

— **mode** pour les variables catégorielles ;

— **KNNImputer** spécifiquement pour `Age`.

4.7 Suppression de variables

Deux types de suppressions ont été effectués :

— colonnes trop manquantes, selon un seuil évalué sur le train ;

— colonnes numériques trop corrélées entre elles.

4.8 Encodage et normalisation

Le prétraitement final repose sur un `ColumnTransformer` combinant :

— `SimpleImputer` + `StandardScaler` pour les variables numériques ;

— `KNNImputer` + `StandardScaler` pour `Age` ;

— `SimpleImputer` + `OneHotEncoder` pour les variables catégorielles.

4.9 Rééquilibrage du dataset

Le rééquilibrage a été réalisé uniquement sur le train via `SMOTENC`. Le jeu d'entraînement initial contient 3497 observations, puis 4668 après sur-échantillonnage. La distribution finale des classes devient équilibrée, avec 2334 observations pour chaque classe.

4.10 Séparation train/test

La séparation train/test a été effectuée avec :

— une proportion de 80/20 ;

— une stratification sur la cible ;

— une reproductibilité assurée par `random_state = 42`.

5 Réduction de dimension et segmentation

5.1 Analyse en composantes principales (PCA)

Une ACP automatique a été appliquée après le prétraitement afin de réduire la dimensionnalité tout en conservant l'essentiel de l'information. Le seuil de variance cumulée retenu a été fixé à 85%.

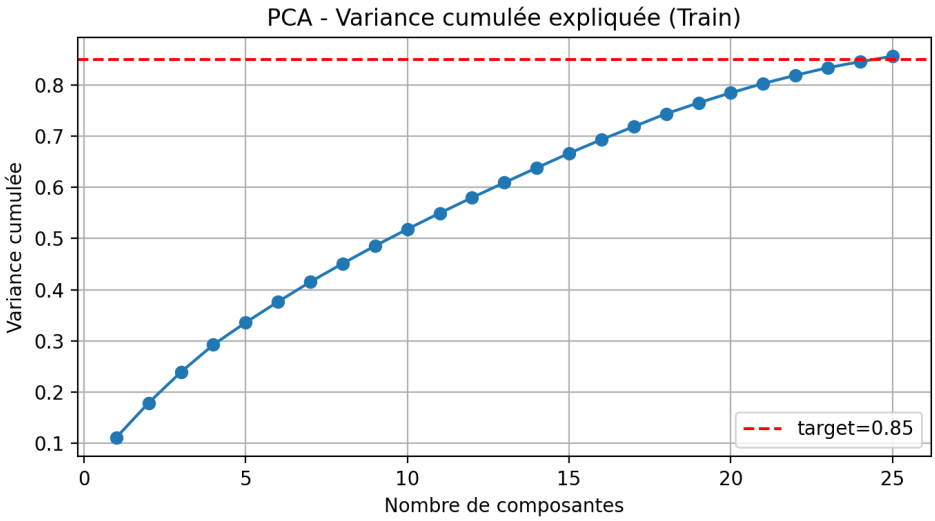

FIGURE 2 – Variance cumulée expliquée par les composantes principales.

5.2 Interprétation de la PCA

La PCA permet à la fois de réduire le bruit, de compresser l'information et de faciliter la visualisation en deux dimensions.

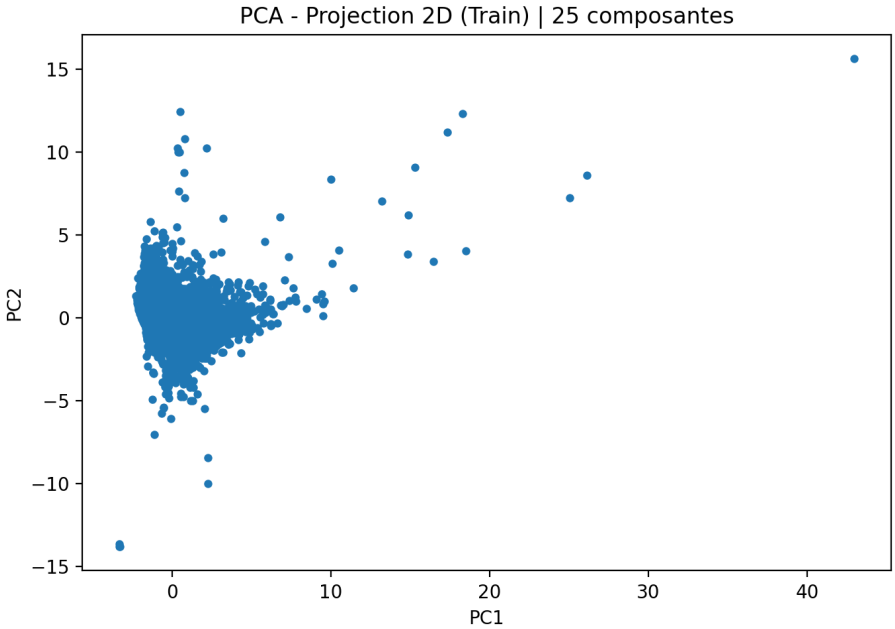

FIGURE 3 – Projection 2D des données après ACP.

5.3 KMeans

Après réduction de dimension, l'algorithme KMeans a été utilisé pour explorer une segmentation non supervisée des clients. Le choix du nombre de clusters s'est appuyé principalement sur le score de silhouette, en complément de l'inertie et d'autres indicateurs.

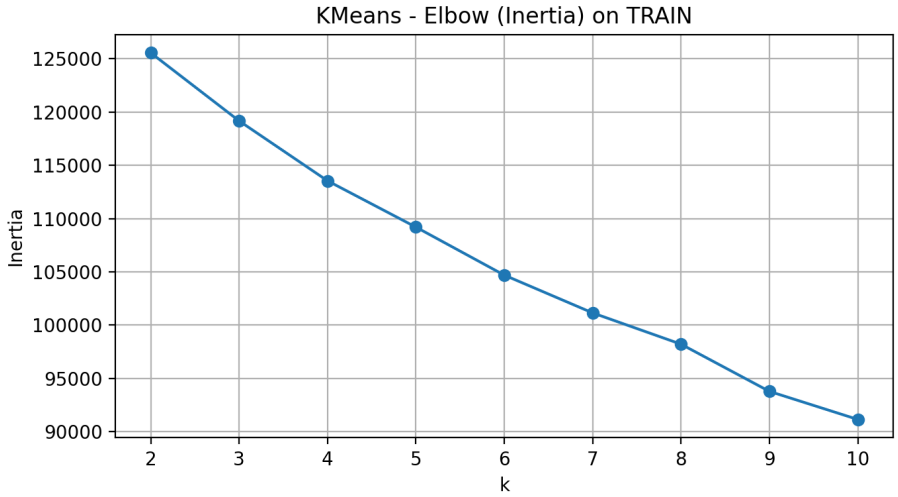

FIGURE 4 – Méthode du coude pour KMeans.

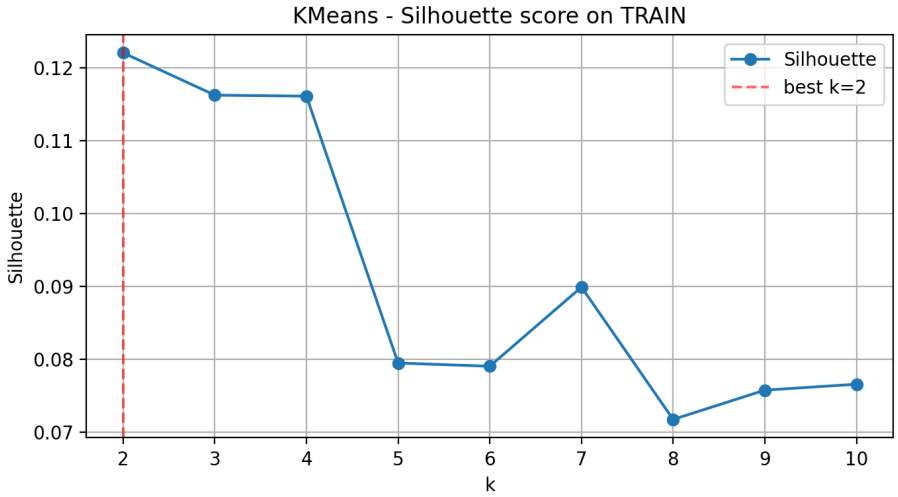

FIGURE 5 – Score de silhouette selon le nombre de clusters.

5.4 Choix final du nombre de clusters

La version finale du code retient automatiquement le nombre de clusters donnant le meilleur score de silhouette. Sur l'exécution finale conservée pour le rapport, le meilleur compromis est obtenu avec $k = 2$. Cela signifie que la segmentation la plus stable et la plus défendable dans ce projet oppose deux grands profils de clients plutôt qu'une partition fine en plusieurs groupes.

5.5 Interprétation des clusters

La projection des clusters sur le plan principal permet de visualiser deux groupes globaux. Cette segmentation a une valeur surtout exploratoire : elle permet de distinguer un groupe central de clients au comportement plus standard et un autre groupe davantage atypique ou dispersé.

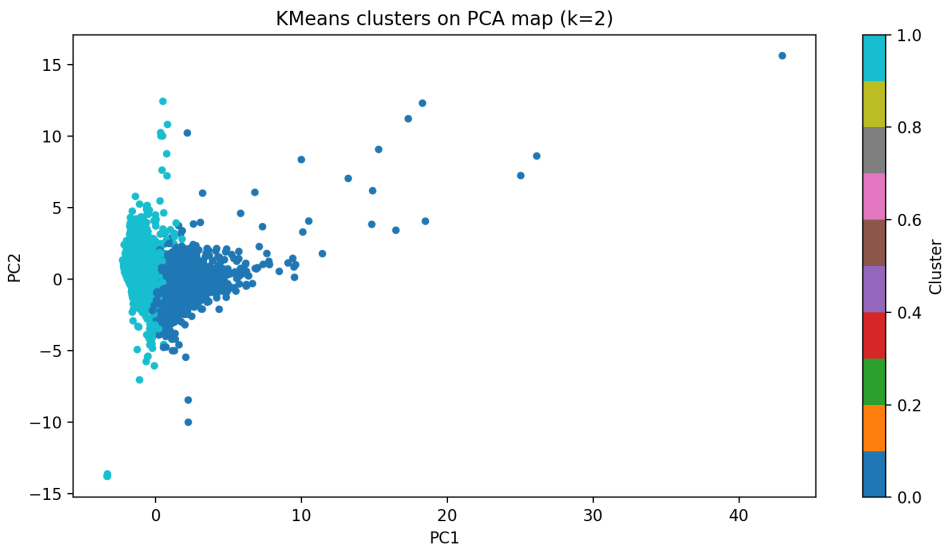

FIGURE 6 – Visualisation des clusters KMeans sur le plan ACP.

5.6 Limites de la segmentation

La segmentation obtenue est utile pour une lecture exploratoire, mais elle reste secondaire par rapport à la classification supervisée du churn. KMeans n'est donc pas ici la composante la plus forte du projet.

6 Modélisation prédictive : classification

6.1 Objectif

L'objectif principal de cette partie est de prédire la probabilité de churn à partir des variables préparées.

6.2 Modèles testés

Trois familles de modèles ont été évaluées :

— **Logistic Regression**

— **SVM**

— **Random Forest**

6.3 Optimisation des hyperparamètres

Le projet ne s'est pas limité à des modèles de base. Une optimisation a été menée par :

— `GridSearchCV` pour Logistic Regression ;

— `GridSearchCV` pour SVM ;

— `GridSearchCV` pour Random Forest ;

— `Optuna` pour affiner la régression logistique.

6.4 Métriques d'évaluation

Les métriques retenues sont :

— Accuracy

— F1-score

— ROC-AUC

— Matrice de confusion

6.5 Résultats obtenus

Le meilleur modèle final est une régression logistique optimisée, avec des performances très élevées :

— Accuracy ≈ 0.994

— F1-score ≈ 0.991

— ROC-AUC ≈ 0.999

TABLE 1 – Synthèse des résultats de classification

Modèle	**Accuracy**	**F1-score**	**ROC-AUC**
LogisticRegression	0.9851	0.9775	0.9981
LogisticRegression_GridSearch	0.9931	0.9897	0.9993
LogisticRegression_Optuna	0.9943	0.9914	0.9993
SVM	0.9726	0.9574	0.9956
SVM_GridSearch	0.9771	0.9654	0.9976
RandomForest_baseline	0.9589	0.9362	0.9908
RandomForest_GridSearch	0.9566	0.9329	0.9900

6.6 Matrice de confusion

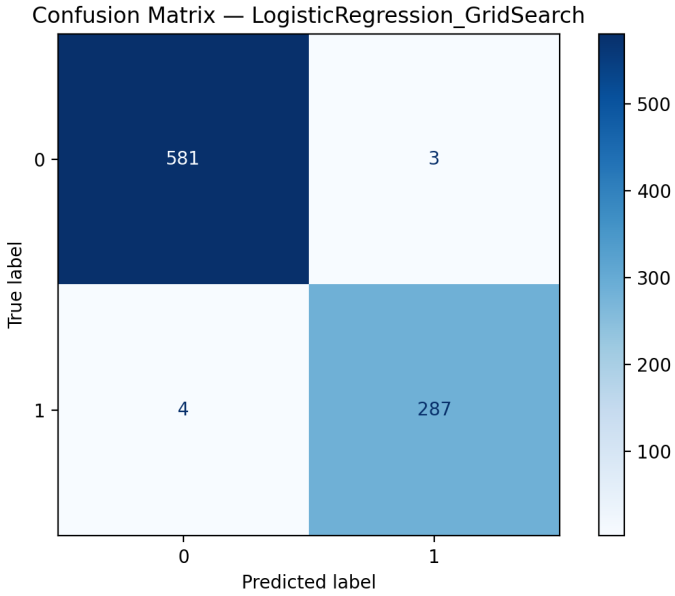

FIGURE 7 – Matrice de confusion du meilleur modèle de classification.

La matrice de confusion finale montre un très faible nombre d'erreurs. Le modèle identifie correctement la grande majorité des clients fidèles et des clients partants, ce qui confirme sa forte capacité de discrimination.

6.7 Analyse critique

La régression logistique donne ici les meilleurs résultats, ce qui montre qu'un modèle relativement simple peut être extrêmement performant lorsque le prétraitement est rigoureux et que les variables sont très informatives. Toutefois, ces performances très élevées doivent être présentées avec prudence, en gardant à l'esprit la nature du dataset et le risque de variables fortement explicatives de la cible.

7 Modélisation prédictive : régression

7.1 Objectif

L'objectif secondaire est de prédire `MonetaryTotal`, c'est-à-dire la valeur monétaire totale associée au client.

7.2 Modèles testés

Deux modèles ont été comparés :

— Ridge

— RandomForestRegressor

7.3 Métriques

L'évaluation repose sur :

— RMSE

— MAE

— R^2

7.4 Résultats

Le meilleur modèle obtenu est `RandomForestRegressor`, avec un pouvoir explicatif nettement supérieur à Ridge.

Table 2 – Résultats de régression

Modèle	**RMSE**	**MAE**	R^2
Ridge	7731.88	1560.31	0.4348
RandomForestRegressor	4939.50	331.68	0.7693

7.5 Analyse critique

La supériorité du RandomForestRegressor suggère que la relation entre les variables explicatives et `MonetaryTotal` est non linéaire. Un modèle linéaire comme Ridge reste plus limité dans ce contexte.

8 Validation croisée et robustesse

8.1 Validation croisée en classification

Une validation croisée stratifiée en 5 plis a été réalisée pour les principaux modèles de classification.

Table 3 – Validation croisée en classification

Modèle	Accuracy	F1-score	ROC-AUC
LogisticRegression	0.9880 ± 0.0025	0.9879 ± 0.0025	0.9992 ± 0.0003
SVM	0.9764 ± 0.0050	0.9762 ± 0.0052	0.9973 ± 0.0008
RandomForest	0.9644 ± 0.0068	0.9641 ± 0.0070	0.9954 ± 0.0010

8.2 Validation croisée en régression

Une validation croisée en 5 plis a également été effectuée pour la régression.

Table 4 – Validation croisée en régression

Modèle	R^2	RMSE
Ridge	0.2844 ± 0.4286	4698.84 ± 1604.82
RandomForestRegressor	0.6879 ± 0.0977	3335.98 ± 1231.76

8.3 Interprétation

Les résultats de validation croisée confirment que les modèles sont globalement stables. La faible variance observée en classification renforce la crédibilité des performances du meilleur classifieur.

9 Prédiction et inférence

9.1 Rôle de `predict.py`

Le script `predict.py` sert à appliquer le modèle entraîné sur de nouvelles données. Il recharge :

- le préprocesseur sauvegardé ;
- le meilleur modèle de classification ;
- les bornes IQR des outliers ;
- la liste des colonnes supprimées au train.

9.2 Pipeline d'inférence

Le pipeline d'inférence reproduit strictement les étapes apprises pendant la phase d'entraînement :

- parsing de la date ;
- feature engineering sur l'adresse IP ;
- suppression des mêmes colonnes inutiles ou fuyantes ;
- réapplication du clipping IQR ;
- alignement exact sur les colonnes attendues par le préprocesseur ;
- prédiction finale du churn et de la probabilité associée.

9.3 Test sur données complètes

Les tests effectués sur de vraies lignes complètes du dataset ont montré que le modèle produit des prédictions cohérentes et variées, ce qui confirme que le pipeline appris fonctionne correctement.

9.4 Limite observée

Une limite importante a été identifiée : lorsque l'interface utilisateur ne fournit qu'un petit sous-ensemble des variables attendues par le modèle complet, de nombreuses colonnes sont imputées automatiquement. Cela peut conduire à des prédictions moins fiables ou artificiellement stables.

10 Déploiement et mise en production avec Flask

10.1 Objectif du déploiement

L'objectif du déploiement est de rendre le modèle de prédiction du churn accessible via une interface web interactive, permettant à un utilisateur non technique de tester différents profils clients et d'obtenir une estimation du risque de churn accompagnée d'une probabilité.

10.2 Architecture de la solution

La solution déployée repose sur une architecture simple mais complète :

— un frontend HTML/CSS pour la saisie des informations ;

— une application Flask jouant le rôle de backend ;

— le préprocesseur sauvegardé ;

— le meilleur modèle de classification entraîné ;

— les artefacts nécessaires au pipeline d'inférence.

10.3 Deux modes de test

L'interface permet deux modes d'utilisation :

— **Mode manuel** : l'utilisateur saisit 10 variables principales ;

— **Mode exact dataset** : l'utilisateur choisit l'index d'une ligne réelle de `X_test.csv`.

Cette double approche permet à la fois une démonstration simple et une vérification fidèle du comportement réel du modèle.

10.4 Fonctionnement détaillé

Dans la version finale, le backend Flask :

— récupère les données du formulaire ;

— normalise les formats numériques ;

— convertit les champs nécessaires en type numérique ;

— reconstruit un DataFrame compatible avec le modèle ;

— applique le même pipeline de prétraitement qu'en phase d'entraînement ;

— renvoie la classe prédite et la probabilité associée.

10.5 Problèmes rencontrés et corrections

Plusieurs difficultés ont été rencontrées au cours du déploiement :

— erreurs de conversion des nombres saisis avec virgule ;

— mauvaise gestion de certaines valeurs manquantes ;

— incohérence entre les variables saisies manuellement et les variables réellement attendues par le modèle ;

— résultats trop extrêmes dans les premières versions de l'interface.

Ces problèmes ont été corrigés en :

— imposant une conversion plus robuste vers les types numériques ;

— alignant explicitement les colonnes avec celles attendues par le préprocesseur ;

— testant l'application avec des lignes réelles issues de `X_test.csv`.

10.6 Résultats observés dans l'interface

Les tests effectués via Flask montrent que le modèle fournit désormais des probabilités réalistes et non seulement des valeurs extrêmes. Par exemple, certaines lignes réelles du jeu de test produisent une probabilité faible de churn, tandis que d'autres donnent une probabilité plus élevée, ce qui confirme le bon fonctionnement du pipeline complet.

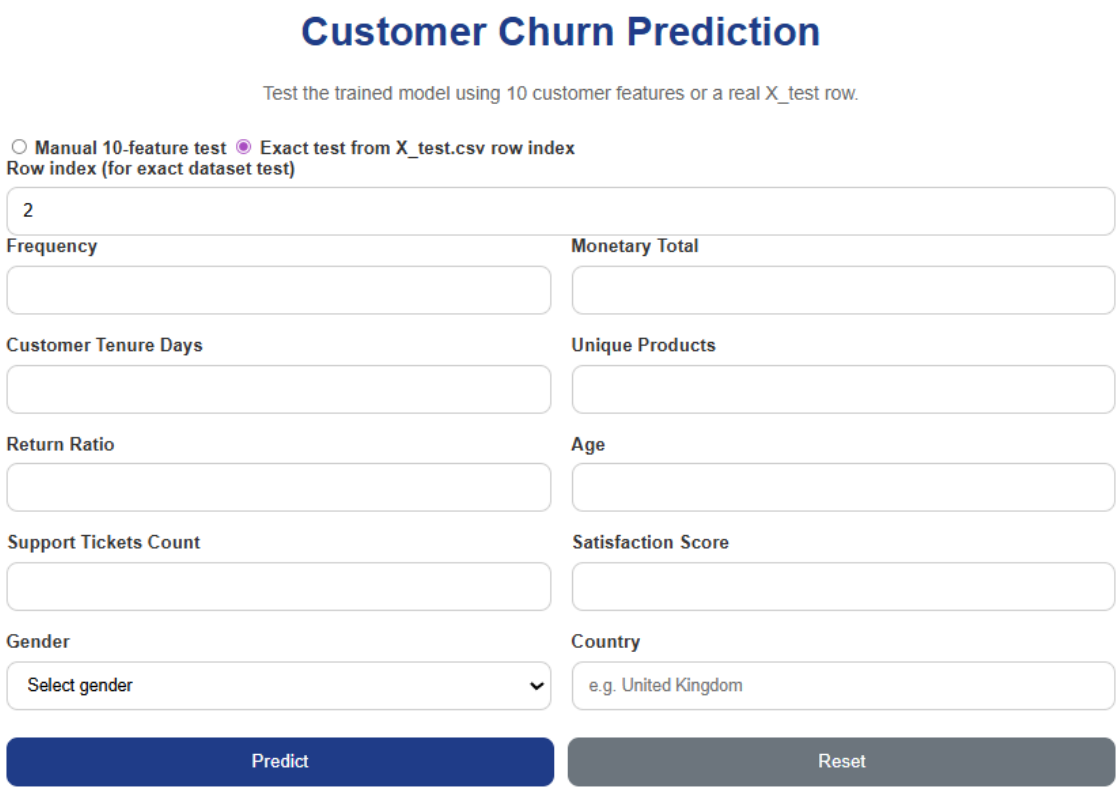

FIGURE 8 – Interface Flask de prédiction du churn.

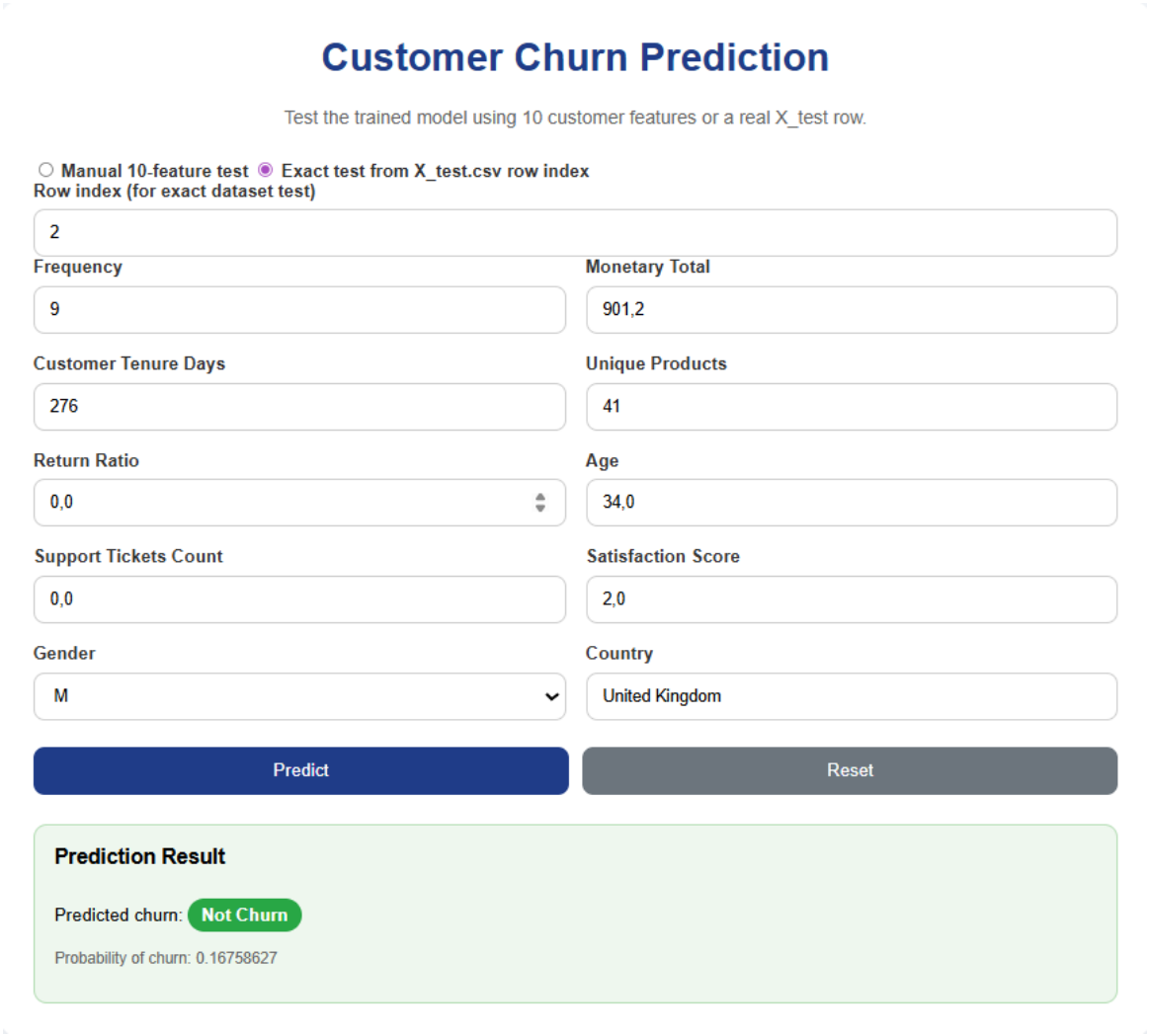

FIGURE 9 – Exemple de prédiction dans Flask : client classé *Not Churn*.

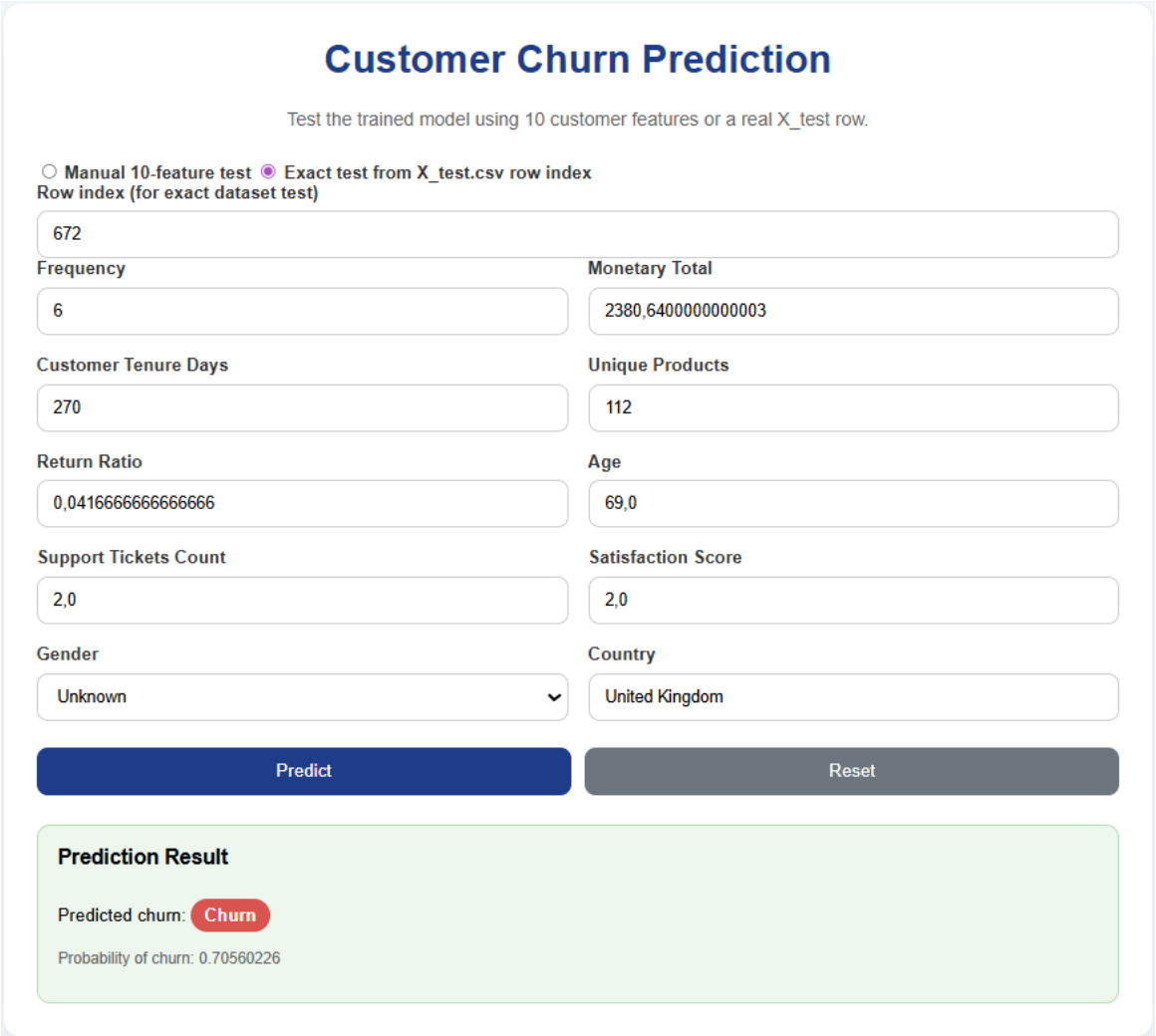

FIGURE 10 – Exemple de prédiction dans Flask : client classé *Churn*.

10.7 Interprétation métier des probabilités

Les probabilités générées par le modèle permettent une lecture plus fine du risque :

— probabilité faible : client relativement stable ;

— probabilité intermédiaire : client à surveiller ;

— probabilité élevée : client à risque élevé de churn.

10.8 Limites du déploiement actuel

L'interface actuelle est volontairement simplifiée. Elle ne couvre pas toutes les variables utilisées par le modèle complet. Ainsi, en mode manuel, certaines colonnes doivent être imputées automatiquement, ce qui peut introduire un écart avec les prédictions obtenues sur des lignes complètes du dataset.

10.9 Améliorations possibles

Pour aller plus loin, plusieurs extensions sont envisageables :

— enrichir le formulaire avec davantage de variables ;

— développer un modèle dédié au mode manuel ;

— intégrer une graduation du risque (faible, moyen, élevé) ;

— ajouter des outils d'explicabilité.

11 Discussion générale

11.1 Points forts du projet

Les principaux points forts sont :

— une chaîne ML complète de bout en bout ;

— une gestion méthodologiquement correcte du data leakage ;

— un preprocessing rigoureux ;

— une comparaison de plusieurs modèles ;

— un tuning avancé via GridSearchCV et Optuna ;

— une validation croisée ;

— une phase d'inférence et de déploiement ;

— une interface Flask fonctionnelle pour tester le modèle.

11.2 Limites

Les limites principales du projet sont :

— le décalage entre le modèle complet et l'interface Flask simplifiée ;

— le caractère surtout exploratoire de la partie PCA/KMeans ;

— la nécessité de rester prudent dans l'interprétation de performances très élevées.

11.3 Améliorations possibles

Plusieurs pistes d'amélioration peuvent être envisagées :

— construire un modèle dédié à l'interface utilisateur ;

— enrichir les entrées du formulaire Flask ;

— intégrer des outils d'interprétabilité tels que SHAP ;

— calibrer les probabilités ;

— tester le pipeline sur un autre jeu de données.

12 Conclusion générale

Ce projet a permis de mettre en œuvre une démarche complète de Machine Learning appliquée à la prédiction du churn client dans un contexte retail. À partir d'un jeu de données riche et imparfait, un pipeline robuste a été construit pour préparer les données, réduire la dimension, segmenter les clients, entraîner plusieurs modèles supervisés, comparer leurs performances et déployer une solution d'inférence.

Le meilleur modèle obtenu est une régression logistique optimisée, offrant d'excellentes performances sur la tâche de classification. En parallèle, un modèle de régression performant a également été mis en place pour prédire la valeur monétaire. Enfin, une interface Flask a été développée afin de démontrer la faisabilité d'un usage applicatif.

L'ensemble du travail montre qu'une préparation rigoureuse des données, une méthodologie soignée et une évaluation sérieuse constituent les fondements d'un projet ML crédible et professionnel.